Java Competitive Programming

Quick Reference · Primitives · Arrays · Collections · Math

1. Primitive Types

Sizes, ranges & literals

byte	8-bit | -128 to 127 | byte b = 127;
short	16-bit | -32,768 to 32,767 | short s = 1000;
int	32-bit | -2^31 to 2^31-1 (~2.1B) | int x = 42;
long	64-bit | -2^63 to 2^63-1 | long l = 9L;
float	32-bit float | float f = 3.14f;
double	64-bit float | double d = 3.14;
char	16-bit Unicode 0-65535 | char c = 'A';
boolean	true / false | boolean ok = true;

Integer / Long constants

Integer.MAX_VALUE	2,147,483,647 (2^31 - 1)
Integer.MIN_VALUE	-2,147,483,648 (-2^31)
Long.MAX_VALUE	9,223,372,036,854,775,807 (2^63 - 1)
Long.MIN_VALUE	-9,223,372,036,854,775,808
Integer.MAX_VALUE/2	Safe "infinity" for DP: 1_073_741_823
(long)1e18	Common INF for long DP problems
Integer.toBinaryString(n)	Convert int to binary string
Integer.bitCount(n)	Count set bits (popcount)
Integer.numberOfLeadingZeros(n)	Leading zeros
Integer.reverse(n)	Reverse bits
Integer.highestOneBit(n)	Highest power-of-2 <= n
Integer.lowestOneBit(n)	Lowest set bit

Primitive arithmetic & casting

```
int a = 5, b = 3;
int sum = a + b; // 8
int diff = a - b; // 2
int prod = a * b; // 15
int quot = a / b; // 1 (integer division)
int rem = a % b; // 2
long big = (long) a * 1_000_000_000; // avoid overflow: cast BEFORE multiply
double d = (double) a / b; // 1.666...
int back = (int) d; // 1 (truncates)
char c = 'A'; int ascii = c; // 65
int dig = c - '0'; // digit char to int
```

Autoboxing — wrapper types

Integer / Long / Double	Wrapper for use in Collections

`Integer.parseInt("42")`	String -> int
`Long.parseLong("99")`	String -> long
`String.valueOf(42)`	int -> String
`Integer.toString(x,2)`	int -> binary string (radix 2)
`Integer.parseInt("1010",2)`	binary string -> int

■ 2. Math Class (java.lang.Math)

Core methods — all static

Math.abs(x)	Absolute value | int/long/double
Math.max(a, b)	Maximum of two values
Math.min(a, b)	Minimum of two values
Math.pow(base, exp)	base^exp -> double
Math.sqrt(x)	Square root -> double
Math.cbrt(x)	Cube root -> double
Math.log(x)	Natural logarithm (base e)
Math.log10(x)	Base-10 logarithm
Math.ceil(x)	Round up -> double
Math.floor(x)	Round down -> double
Math.round(x)	Round to nearest long/int
Math.hypot(x, y)	sqrt(x^2 + y^2) — safe, no overflow
Math.PI	3.141592653589793
Math.E	2.718281828459045
Math.signum(x)	-1.0 / 0.0 / 1.0
Math.floorMod(a, b)	Always non-negative modulo
Math.floorDiv(a, b)	Floor division (like Python //)
Math.gcd not built-in	-> see snippet below

Essential CP snippets

```
// Integer square root (exact)
long isqrt(long n) { return (long) Math.sqrt(n); }

// GCD (iterative)
long gcd(long a, long b) { while(b!=0){long t=b;b=a%b;a=t;} return a; }

// LCM
long lcm(long a, long b) { return a / gcd(a,b) * b; }

// Fast power (mod)
long powmod(long b, long e, long mod) {
long r=1; b%=mod;
while(e>0){ if((e&1)==1) r=r*b%mod; b=b*b%mod; e>>=1; }
return r;
}

// Log2 (integer)
int log2(int n) { return 31 - Integer.numberOfLeadingZeros(n); }

// isPowerOfTwo
boolean isPow2(int n) { return n>0 && (n & (n-1))==0; }

// Overflow-safe add check
boolean addOverflows(long a, long b) {
return (b>0 && a > Long.MAX_VALUE-b) || (b<0 && a < Long.MIN_VALUE-b);
}
```

Bit tricks

`n & 1`	Check if odd
`n >> 1`	Divide by 2 (arithmetic shift)
`n << 1`	Multiply by 2
`n & (n-1)`	Clear lowest set bit
`n | (1 << k)`	Set k-th bit
`n & ~(1 << k)`	Clear k-th bit
`n ^ (1 << k)`	Toggle k-th bit
`(n >> k) & 1`	Get k-th bit
`n & -n`	Isolate lowest set bit (Fenwick trick)
`~n + 1`	Two's complement negation

3. Arrays

Declaration & initialization

```
int[] a = new int[n]; // zeros
int[] a = {1, 2, 3, 4}; // literal
int[] a = new int[]{5, 6, 7}; // anonymous
int[][] m = new int[rows][cols]; // 2-D, zeros
int[][] m = {{1,2},{3,4}}; // 2-D literal
long[] l = new long[n];
char[] c = new char[n];
boolean[]fl = new boolean[n];
String[] s = new String[n];

// Fill with value
Arrays.fill(a, -1);
Arrays.fill(m[i], 0); // fill one row
```

java.util.Arrays — essential methods

Arrays.sort(a)	Sort int[] ascending O(n log n)
Arrays.sort(a, l, r)	Sort subarray [l, r)
Arrays.sort(a, Comparator)	Sort Object[] with comparator (not int[])
Arrays.fill(a, v)	Fill entire array with v
Arrays.fill(a, l, r, v)	Fill subarray [l,r) with v
Arrays.copyOf(a, len)	New array, first len elements
Arrays.copyOfRange(a, l, r)	New array from [l, r)
Arrays.equals(a, b)	Element-wise equality
Arrays.binarySearch(a, key)	Binary search on SORTED array -> index
Arrays.toString(a)	For printing: "[1, 2, 3]"
Arrays.deepToString(m)	For 2-D: "[[1,2],[3,4]]"
Arrays.stream(a).sum()	Sum all elements
Arrays.stream(a).max().getAsInt()	Max element
Arrays.stream(a).min().getAsInt()	Min element

Common array patterns

```
// Reverse an array
for(int i=0,j=a.length-1; i<j; i++,j--){ int t=a[i];a[i]=a[j];a[j]=t; }

// Prefix sum
long[] pre = new long[n+1];
for(int i=0;i<n;i++) pre[i+1]=pre[i]+a[i];
long rangeSum = pre[r+1] - pre[l]; // sum [l..r] inclusive

// 2-D prefix sum
int[][] ps = new int[n+1][m+1];
for(int i=1;i<=n;i++) for(int j=1;j<=m;j++)
ps[i][j]=a[i-1][j-1]+ps[i-1][j]+ps[i][j-1]-ps[i-1][j-1];
```

```
// Coordinate compression
int[] sorted = a.clone(); Arrays.sort(sorted);
sorted = Arrays.stream(sorted).distinct().toArray();
Map<Integer,Integer> compress = new HashMap<>();
for(int i=0;i<sorted.length;i++) compress.put(sorted[i],i);

// Sort descending (Integer array)
Integer[] b = Arrays.stream(a).boxed().toArray(Integer[]::new);
Arrays.sort(b, (x,y)->y-x);

// Sort int[][] by first column
Arrays.sort(intervals, (x,y)->x[0]-y[0]);
```

Lower bound / upper bound on sorted int[]

```
// Lower bound: first index >= key
int lowerBound(int[] a, int key) {
int lo=0, hi=a.length;
while(lo<hi){ int mid=(lo+hi)>>>1; if(a[mid]<key) lo=mid+1; else hi=mid; }
return lo;
}

// Upper bound: first index > key
int upperBound(int[] a, int key) {
int lo=0, hi=a.length;
while(lo<hi){ int mid=(lo+hi)>>>1; if(a[mid]<=key) lo=mid+1; else hi=mid; }
return lo;
}

// Using Arrays.binarySearch (sorted array):
int idx = Arrays.binarySearch(a, key);
// if idx < 0: insertion point = -(idx+1)
int insertPos = idx >= 0 ? idx : -(idx+1);
```

■ 4. Collections Framework

ArrayList — dynamic array, O(1) get/set, O(n) insert/remove

```
List<Integer> list = new ArrayList<>();
List<Integer> list = new ArrayList<>(Arrays.asList(1,2,3));
List<Integer> list = new ArrayList<>(n); // initial capacity hint
```

`list.add(e)`	Append to end O(1) amortized
`list.add(i, e)`	Insert at index i O(n)
`list.get(i)`	Get element at i O(1)
`list.set(i, e)`	Update element at i O(1)
`list.remove(i)`	Remove by index O(n)
`list.remove((Integer)v)`	Remove first occurrence of value
`list.size()`	Number of elements
`list.isEmpty()`	true if empty
`list.contains(e)`	Linear search O(n)
`list.indexOf(e)`	First index of e (-1 if absent)
`list.clear()`	Remove all elements
`list.subList(l, r)`	View [l, r) — modifications reflect back
`Collections.sort(list)`	Sort ascending
`Collections.sort(list, Collections.reverseOrder())`	Sort descending
`Collections.reverse(list)`	Reverse in-place
`Collections.shuffle(list)`	Shuffle randomly
`Collections.frequency(list, e)`	Count occurrences
`Collections.min(list)`	Minimum element
`Collections.max(list)`	Maximum element
`Collections.fill(list,v)`	Fill all positions with v
`Collections.nCopies(n,v)`	Immutable list of n copies of v
`list.toArray(new Integer[0])`	Convert to array

Lower / upper bound on sorted ArrayList

```
Collections.sort(list);

// Lower bound: first index >= key
int lb = Collections.binarySearch(list, key);
if(lb < 0) lb = -(lb+1); // insertion point when not found
// lb is now the index of first element >= key

// Upper bound: first index > key
int ub = Collections.binarySearch(list, key);
if(ub < 0) ub = -(ub+1);
else { while(ub < list.size() && list.get(ub).equals(key)) ub++; }
```

LinkedList — deque operations, O(1) head/tail

```
Deque<Integer> dq = new LinkedList<>();
LinkedList<Integer> ll = new LinkedList<>();
```

ll.addFirst(e) / offerFirst(e)	Push to front
ll.addLast(e) / offerLast(e)	Push to back
ll.removeFirst() / pollFirst()	Pop from front (null-safe: poll)
ll.removeLast() / pollLast()	Pop from back
ll.peekFirst() / peekLast()	Peek without removing
ll.getFirst() / getLast()	Peek (throws if empty)
ll.size(), ll.isEmpty()	Size checks

■ 5. Stack · Queue · Deque · PriorityQueue

Stack (prefer Deque<> over legacy Stack<>)

```
Deque<Integer> stk = new ArrayDeque<>();
stk.push(x); // addFirst
stk.pop(); // removeFirst
stk.peek(); // peekFirst
stk.isEmpty();
```

Queue (FIFO)

```
Queue<Integer> q = new LinkedList<>();
Queue<Integer> q = new ArrayDeque<>(); // faster
q.offer(x); // enqueue (add to tail)
q.poll(); // dequeue (remove from head) null if empty
q.peek(); // front element, null if empty
q.isEmpty(); q.size();
```

Deque (double-ended queue)

dq.offerFirst(x)	Push front
dq.offerLast(x)	Push back
dq.pollFirst()	Pop front (null-safe)
dq.pollLast()	Pop back (null-safe)
dq.peekFirst()	Front peek
dq.peekLast()	Back peek
dq.size()	Number of elements

PriorityQueue (min-heap by default)

```
PriorityQueue<Integer> minH = new PriorityQueue<>();
PriorityQueue<Integer> maxH = new PriorityQueue<>(Collections.reverseOrder());

// Custom comparator (e.g. sort by second element of int[])
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[1]-b[1]);

pq.offer(x); // insert O(log n)
pq.poll(); // remove & return min O(log n)
pq.peek(); // view min O(1)
pq.size(); pq.isEmpty();

// Dijkstra pattern
pq.offer(new int[]{dist, node});
int[] cur = pq.poll(); // cur[0]=dist, cur[1]=node
```

ArrayDeque as monotonic deque (sliding window max)

```
Deque<Integer> mono = new ArrayDeque<>(); // stores indices
for(int i=0;i<n;i++){
while(!mono.isEmpty() && a[mono.peekLast()] <= a[i]) mono.pollLast();
mono.offerLast(i);
if(mono.peekFirst() <= i-k) mono.pollFirst();
if(i>=k-1) ans[i-k+1] = a[mono.peekFirst()];
```

```
}
```

6. Map — HashMap · TreeMap · LinkedHashMap

HashMap — O(1) average, unordered

```
Map<Integer,Integer> map = new HashMap<>();
Map<String,List<Integer>> map = new HashMap<>();
```

`map.put(k, v)`	Insert / overwrite
`map.get(k)`	Get value (null if absent)
`map.getOrDefault(k, def)`	Get or return default
`map.containsKey(k)`	O(1) key lookup
`map.containsValue(v)`	O(n) value lookup
`map.remove(k)`	Delete entry
`map.size() map.isEmpty()`	Size checks
`map.keySet()`	Set of all keys
`map.values()`	Collection of all values
`map.entrySet()`	Set> for iteration
`map.putIfAbsent(k, v)`	Insert only if key absent
`map.merge(k, 1, Integer::sum)`	Increment count (freq map)
`map.compute(k, (k,v)->...)`	Compute new value in-place
`map.forEach((k,v)->...)`	Iterate key-value pairs
`map.getOrDefault(k,0)+1`	Classic frequency increment pattern

TreeMap — O(log n), sorted by key

`tm.firstKey() / lastKey()`	Min / max key
`tm.floorKey(k)`	Largest key <= k (lower_bound analog)
`tm.ceilingKey(k)`	Smallest key >= k
`tm.lowerKey(k)`	Largest key < k
`tm.higherKey(k)`	Smallest key > k
`tm.floorEntry(k)`	Map.Entry with largest key <= k
`tm.headMap(k)`	View of keys < k
`tm.tailMap(k)`	View of keys >= k
`tm.subMap(lo, hi)`	View of keys in [lo, hi)
`tm.pollFirstEntry()`	Remove & return smallest entry
`tm.pollLastEntry()`	Remove & return largest entry
`tm.descendingMap()`	Reverse-order view

LinkedHashMap — insertion-order iteration

```
// LRU cache (access-order)
Map<Integer,Integer> lru = new LinkedHashMap<>(16, 0.75f, true) {
protected boolean removeEldestEntry(Map.Entry e) { return size()>cap; }
};
```

7. Set — HashSet · TreeSet · LinkedHashSet

HashSet — O(1) average, unordered

```
Set<Integer> set = new HashSet<>();
Set<Integer> set = new HashSet<>(Arrays.asList(1,2,3));
```

`set.add(e)`	Insert O(1)
`set.remove(e)`	Delete O(1)
`set.contains(e)`	Membership test O(1)
`set.size()`	Number of elements
`set.isEmpty()`	True if empty
`set.clear()`	Remove all
`set.addAll(col)`	Union with collection
`set.retainAll(col)`	Intersection in-place
`set.removeAll(col)`	Difference in-place

TreeSet — O(log n), sorted, like C++ std::set

`ts.first() / last()`	Min / max element
`ts.floor(e)`	Largest element <= e (lower_bound)
`ts.ceiling(e)`	Smallest element >= e
`ts.lower(e)`	Largest element < e
`ts.higher(e)`	Smallest element > e
`ts.headSet(e)`	View of elements < e
`ts.tailSet(e)`	View of elements >= e
`ts.subSet(lo, hi)`	View [lo, hi)
`ts.pollFirst()`	Remove & return smallest
`ts.pollLast()`	Remove & return largest
`ts.descendingSet()`	Reverse-order view
`ts.size()`	Number of elements

Common set patterns

```
// Frequency map -> distinct count
Set<Integer> distinct = new HashSet<>(Arrays.asList(arr));
int numDistinct = distinct.size();

// Check duplicate in array
Set<Integer> seen = new HashSet<>();
for(int x : arr) if(!seen.add(x)) return true;

// Complement / sliding window with set
Set<Integer> window = new HashSet<>();
int l=0;
for(int r=0;r<n;r++){
while(window.contains(a[r])) window.remove(a[l++]);
window.add(a[r]);
ans = Math.max(ans, r-l+1);
}
```

8. String & StringBuilder

String — immutable

`s.length()`	Length
`s.charAt(i)`	Character at index i
`s.indexOf(c)`	First occurrence of char/str (-1 if absent)
`s.lastIndexOf(c)`	Last occurrence
`s.substring(l, r)`	Substring [l, r)
`s.substring(l)`	Suffix from l
`s.toLowerCase() / toUpperCase()`	Case conversion
`s.trim()`	Strip leading/trailing whitespace
`s.strip()`	Unicode-aware trim (Java 11+)
`s.split(regex)`	Split into String[]
`s.split(" ", -1)`	-1 keeps trailing empty strings
`s.replace(old, new)`	Replace all occurrences
`s.replaceAll(regex, rep)`	Regex replace
`s.contains(sub)`	Substring check
`s.startsWith(pre)`	Prefix check
`s.endsWith(suf)`	Suffix check
`s.equals(t)`	Content equality (NOT ==)
`s.equalsIgnoreCase(t)`	Case-insensitive equality
`s.compareTo(t)`	Lexicographic comparison
`s.toCharArray()`	Convert to char[]
`String.valueOf(x)`	int/long/char -> String
`String.join(", ", list)`	Join with delimiter
`"".join("", arr)`	Concatenate array
`s.chars()`	IntStream of char values
`s.repeat(n)`	Repeat string n times (Java 11+)
`s.formatted(args)`	Like String.format (Java 15+)

StringBuilder — mutable, use for building strings in loops

`new StringBuilder()`	Empty builder
`new StringBuilder(s)`	Start from String
`sb.append(x)`	Append any type — returns sb (chainable)
`sb.insert(i, x)`	Insert at index i
`sb.delete(l, r)`	Delete [l, r)
`sb.deleteCharAt(i)`	Delete single char
`sb.replace(l, r, s)`	Replace [l,r) with s
`sb.reverse()`	Reverse in-place
`sb.charAt(i)`	Get char
`sb.setCharAt(i, c)`	Set char
`sb.indexOf(s)`	Find substring
`sb.length()`	Current length
`sb.toString()`	Convert to String

Common string patterns

```
// Check palindrome
String rev = new StringBuilder(s).reverse().toString();
boolean isPalin = s.equals(rev);

// Char frequency array
int[] freq = new int[26];
for(char c : s.toCharArray()) freq[c-'a']++;

// Convert char[] back to String
String t = new String(charArr);

// Sort characters of a string
char[] ch = s.toCharArray(); Arrays.sort(ch); String sorted = new String(ch);
```

■ 9. Sorting Cheatsheet & Comparators

Sort primitives

```
int[] a = {3,1,4,1,5};
Arrays.sort(a); // ascending
Arrays.sort(a, 2, 5); // subarray [2,5)

// Descending: must use Integer[]
Integer[] b = {3,1,4,1,5};
Arrays.sort(b, (x,y) -> y-x); // descending
Arrays.sort(b, Collections.reverseOrder());

// Convert int[] <-> Integer[]
Integer[] boxed = Arrays.stream(a).boxed().toArray(Integer[]::new);
int[] unboxed = Arrays.stream(boxed).mapToInt(Integer::intValue).toArray();
```

Comparator patterns

```
// Sort 2-D array by col 0 asc, col 1 desc
Arrays.sort(arr, (x,y) -> x[0]!=y[0] ? x[0]-y[0] : y[1]-x[1]);

// Comparator.comparing (String[])
Arrays.sort(people, Comparator.comparing((String[] p)->p[1])
.thenComparingInt(p->Integer.parseInt(p[0])));

// Sort List<int[]> by element
list.sort((a,b) -> Integer.compare(a[0], b[0]));

// Natural order vs reverse
Comparator<Integer> asc = Integer::compareTo;
Comparator<Integer> desc = Comparator.reverseOrder();

// Caution: use Integer.compare(a,b) not a-b (overflow for negatives)
```

Collections sort utilities

Collections.sort(list)	Sort List ascending
Collections.sort(list, cmp)	Sort with Comparator
Collections.reverse(list)	Reverse List in-place
Collections.shuffle(list)	Shuffle randomly
Collections.binarySearch(list, key)	Binary search on sorted list
Collections.min(list) / max(list)	Min/max in O(n)
Collections.swap(list, i, j)	Swap two elements
Collections.rotate(list, dist)	Rotate by dist positions
Collections.disjoint(c1, c2)	True if no common elements

TreeMap / TreeSet as ordered structures (lower/upper bound summary)

TreeSet.floor(x)	Largest element <= x (C++ lower_bound - 1 analog)
TreeSet.ceiling(x)	Smallest element >= x (C++ lower_bound analog)
TreeSet.lower(x)	Largest element < x (C++ lower_bound - 1 strict)

TreeSet.higher(x)	Smallest element > x (C++ upper_bound analog)
TreeMap.floorKey(x)	Largest key <= x
TreeMap.ceilingKey(x)	Smallest key >= x
TreeMap.lowerKey(x)	Largest key < x
TreeMap.higherKey(x)	Smallest key > x

■ 10. Fast I/O & CP Templates

Fast input — BufferedReader (10x faster than Scanner)

```
import java.io.*;
import java.util.*;

public class Main {
static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
static StringTokenizer st;

static String next() throws IOException {
while(st==null||!st.hasMoreTokens())
st=new StringTokenizer(br.readLine());
return st.nextToken();
}
static int nextInt() throws IOException { return Integer.parseInt(next()); }
static long nextLong() throws IOException { return Long.parseLong(next()); }
static double nextDouble() throws IOException { return Double.parseDouble(next()); }

public static void main(String[] args) throws IOException {
int n = nextInt();
int[] a = new int[n];
for(int i=0;i<n;i++) a[i]=nextInt();
// ...
}
}
```

Fast output — StringBuilder / PrintWriter

```
PrintWriter pw = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
pw.println(ans);
pw.flush(); // MUST flush at end

// Or batch with StringBuilder
StringBuilder sb = new StringBuilder();
for(int i=0;i<n;i++) sb.append(a[i]).append(' ');
System.out.print(sb);
```

DSU / Union-Find

```
int[] par, rnk;
void init(int n){ par=new int[n]; rnk=new int[n];
for(int i=0;i<n;i++) par[i]=i; }
int find(int x){ return par[x]==x?x:(par[x]=find(par[x])); }
boolean union(int a,int b){
a=find(a);b=find(b); if(a==b)return false;
if(rnk[a]<rnk[b]){int t=a;a=b;b=t;}
par[b]=a; if(rnk[a]==rnk[b])rnk[a]++;
return true; }
```

Fenwick Tree (BIT) — point update, prefix sum query

```
long[] bit; int N;
void update(int i, long v){ for(;i<=N;i+=i&-i) bit[i]+=v; }
long query(int i){ long s=0; for(;i>0;i-=i&-i) s+=bit[i]; return s; }
long range(int l, int r){ return query(r)-query(l-1); } // 1-indexed
```

Useful Java gotchas for CP

int overflow	Use long when product/sum may exceed 2^31
a-b comparator	Use Integer.compare(a,b) — a-b overflows for large negatives
Scanner vs BR	BufferedReader is ~10x faster; use for large input
HashMap capacity	new HashMap<>(1<<16) reduces rehashing
List.of() immutable	Cannot add/remove; use new ArrayList<>(List.of(...))
char arithmetic	'a'+1 is int 98, cast: (char)('a'+1) = 'b'
% negative	Java % can return negative: use ((a%m)+m)%m
Integer cache	== works for Integer -128..127 only; always use .equals()
Arrays.sort stable?	For objects yes; for primitives uses dual-pivot quicksort (not stable)
Math.pow precision	Returns double; cast (long)Math.pow(2,30) may be off by 1 near powers